```
  puts(line)  if line =~ /third/ .. line =~ /fifth/
end
```

produces:

```
third
fourth
fifth
```

The start and end of a range used in a boolean expression can themselves be expressions. These are evaluated each time the overall boolean expression is evaluated. For example, the following code uses the fact that the variable $. contains the current input line number to display the first three lines as well as those lines between a match of /eig/ and /nin/:

```
File.foreach("ordinal") do |line|
  if (($. == 1) || line =~ /eig/) .. (($. == 3) || line =~ /nin/)
    print line
  end
end
```

produces:

```
first
second
third
eighth
ninth
```

You'll come across a wrinkle when you use while and until as statement modifiers. If the statement they are modifying is a begin…end block, the code in the block will always execute at least one time, regardless of the value of the boolean expression:

```
print "Hello\n" while false
begin
  print "Goodbye\n"
end while false
```

produces:

```
Goodbye
```

Iterators

If you read the beginning of the previous section, you may have been discouraged. "Ruby has primitive built-in looping constructs," it said. Don't despair, gentle reader, for we have good news. Ruby doesn't need sophisticated built-in loops, because all the fun stuff is implemented using Ruby's iterators.

As we'll see, even Ruby's for loop is defined in terms of Ruby iterators.

Ruby uses methods defined in various built-in classes to provide equivalent, but less error-prone functionality to other languages primitive for loops.

Let's look at some examples:

```
3.times do
  print "Ho! "
end
```